In this chapter, you'll see:

- How to install RSpec and write your first spec
- How to organize your specifications using `describe` and `it`
- How to verify desired outcomes with `expect`
- How to interpret test failures
- How to keep your specs free of repeated setup code

CHAPTER 1

Getting Started With RSpec

RSpec 3 is a productive Ruby test framework. We say *productive* because everything about it—its style, API, libraries, and settings—are designed to support *you* as you write great software.

Writing effective tests helps you toward that goal of shipping your application. We have a specific definition of *effective* here: *does this test pay for the cost of writing and running it?* A good test will provide at least one of these benefits:

- Design guidance: helping you distill all those fantastic ideas in your head into running, maintainable code
- Safety net: finding errors in your code before your customers do
- Documentation: capturing the behavior of a working system to help its maintainers

As you follow along through the examples in this book, you'll practice several habits that will help you test effectively:

- When you describe precisely what you want your program to do, you avoid being too strict (and failing when an irrelevant detail changes) or too lax (and getting false confidence from incomplete tests).
- By writing your specs to report failure at the right level of detail, you give just enough information to find the cause of a problem—without drowning in excessive output.
- By clearly separating essential test code from noisy setup code, you communicate what's actually expected of the application—and you avoid repeating unnecessary detail.
- When you reorder, profile, and filter your specs, you unearth order dependencies, slow tests, and incomplete work.